

MLFF-DATA9B3A Specification: cuEquivariance Campaign Backend

mdstats project

2026-08-04

Contents

1	Scope	1
2	Configuration contract	1
3	Immutable policy and runtime evidence	2
4	Doctor qualification	2
5	End-to-end propagation	2
6	Saved-model portability	3
7	Failure semantics	3
8	Acceptance tests	3

1 Scope

MLFF-DATA9B3A makes the MACE acceleration backend an explicit, immutable campaign policy. It supports ordinary Torch/e3nn execution and the optional NVIDIA cuEquivariance backend without allowing environment-dependent silent fallback after campaign initialization.

The policy applies to foundation inference, DATA6 descriptors and predictions, DATA8 training jobs, the real one-epoch preflight, production training, checkpoint evaluation, and bounded Python/ASE verification. MACE continues to own neural-network execution and optimization; **mdstats** owns backend selection, qualification, propagation, provenance, and promotion gates.

2 Configuration contract

A campaign configuration SHALL contain a resolved backend:

```
[acceleration]
backend = "cueq"           # or "e3nn"
only_cueq = false
require_available = true
```

`init` SHALL inspect the active Python/CUDA environment once. It SHALL write `backend = "cueq"` only when all of the following are available:

1. CUDA is usable when the campaign device is CUDA;
2. `cuequivariance` imports;
3. `cuequivariance_torch` imports;
4. `cuequivariance_ops_torch` imports;
5. the installed MACE calculator exposes `enable_cueq`.

Otherwise `init` SHALL write `backend = "e3nn"`. The active configuration SHALL NOT retain `backend = "auto"`; automatic resolution after initialization would permit the scientific protocol to change when the environment changes.

`only_cueq = true` is not production-qualified in this stage. Production CuEq training SHALL use `only_cueq = false`, allowing MACE to convert saved checkpoints back to portable e3nn-form models.

3 Immutable policy and runtime evidence

`MaceAccelerationPolicy` SHALL own:

- resolved backend;
- `enable_cueq` realization;
- `only_cueq` realization;
- fail-closed availability policy;
- deterministic policy digest.

The acceleration policy SHALL be included in `MaceOptimizerPolicy` and therefore in every complete `TrainingProtocolIdentity` and `DATA8` job identity.

`MaceAccelerationProbe` SHALL record:

- Torch and Torch-CUDA versions;
- CUDA availability;
- MACE version;
- CuEq Python-package versions;
- MACE calculator flag support;
- whether a real model smoke was attempted;
- finite energy, force, and stress evidence;
- explicit error type and message on failure.

The probe digest SHALL be bound to `TrainingCampaignPolicy`. Changing the backend, CuEq package stack, Torch/CUDA runtime, or successful qualification evidence SHALL make prior campaign stages stale rather than silently reusing them.

4 Doctor qualification

For an e3nn campaign, `doctor` SHALL verify the requested device and ordinary MACE environment.

For a CuEq campaign, `doctor` SHALL additionally:

1. import the complete CuEq stack;
2. load the exact foundation checkpoint;
3. load one real replay or target configuration;
4. instantiate `MACECalculator(..., enable_cueq=True)`;
5. evaluate energy, forces, and stress;
6. require every result to be finite;
7. persist the probe record.

If `require_available = true`, any failed CuEq check SHALL block preparation. No implicit e3nn fallback is permitted. To use e3nn, the user SHALL edit the configuration explicitly and rerun `doctor`.

5 End-to-end propagation

The same frozen acceleration policy SHALL be used by:

Campaign operation	Required realization
DATA6 foundation sweep	<code>MACECalculator(enable_cueq=...)</code>
DATA8 job generation	YAML <code>enable_cueq</code> and <code>only_cueq</code>
MACE parser realization	parsed flags equal immutable policy

Campaign operation	Required realization
one-epoch preflight training	generated CuEq-enabled job
preflight evaluation	<code>mdstats-mace-eval --enable_cueq</code>
production training	<code>mdstats-mace-train</code> with frozen YAML
checkpoint evaluation	<code>MACECalculator(enable_cueq=...)</code>
bounded NVE verification	<code>MACECalculator(enable_cueq=...)</code>

A CuEq preflight SHALL also retain log evidence that MACE converted the model to CuEq for accelerated training. Merely importing the packages is not sufficient.

6 Saved-model portability

With `only_cueq = false`, CuEq is an execution backend rather than a new model identity. MACE converts the trained model back to ordinary e3nn form before saving. The exported target-head committee therefore remains loadable by the qualified standard MACE path.

Traditional converted `pair_style mace` LAMMPS deployment is a separate backend and SHALL NOT be claimed as CuEq-accelerated by this policy. Any ML-IAP/CuEq LAMMPS path requires separate deployment qualification.

7 Failure semantics

The campaign SHALL fail closed when:

- CuEq is requested but any required module is absent;
- CUDA is requested but unavailable;
- the MACE calculator does not support the flag;
- real-model conversion or inference fails;
- energy, force, or stress is non-finite;
- DATA8 flags differ from the frozen policy;
- the preflight log lacks CuEq conversion evidence;
- a later execution environment does not match the qualified policy lineage.

No failure SHALL silently change the backend.

8 Acceptance tests

DATA9B3A is accepted when focused tests establish:

- policy serialization and digest stability;
- automatic initialization chooses CuEq only for a qualified environment;
- active configurations reject `auto`;
- DATA8 YAML contains exact CuEq flags;
- MACE parser realization matches the immutable optimizer policy;
- DATA6 provider receives `enable_cueq=True`;
- checkpoint evaluation and bounded NVE receive the same backend;
- campaign identity binds the acceleration-probe digest;
- legacy e3nn protocols deserialize without changing historical digests;
- architecture dependency graph remains acyclic;
- ordinary DATA8-DATA9B regression tests remain clean.